

SLM Studio

API Documentation

Full Reference: Supabase Internal APIs + External Fine-Tuning Engine

Version 2.0 | 2025

engine.slmstudio.ai/v1 & api.slmstudio.ai/v1

Project	Smart Model Tune (SLM Studio)
Stack	React 18 + TypeScript + Vite + Supabase + shadcn/ui
Backend	Supabase BaaS (PostgreSQL + Auth + Row-Level Security)
Engine	engine.slmstudio.ai/v1 (Auto Fine-Tuning Pipeline)
Inference	api.slmstudio.ai/v1 (OpenAI-Compatible Inference)
Auth	Bearer sk-slm-{prefix}-{random4} Supabase JWT
Date	May 2025

Table of Contents

1. Overview & Architecture
2. Authentication
3. Internal Supabase APIs
 - 3.1 Projects API
 - 3.2 Models API
 - 3.3 Datasets API
 - 3.4 Deployments API
 - 3.5 API Keys API
 - 3.6 Analytics API
4. External Fine-Tuning Engine APIs (engine.slmstudio.ai/v1)
 - 4.1 Dataset Upload & Analysis
 - 4.2 Training Job Lifecycle
 - 4.3 Training Progress Stream (SSE)
 - 4.4 Model Evaluation
 - 4.5 Hyperparameter Optimisation (HPO)
 - 4.6 Auto-Tuning Parameters
5. Inference API (api.slmstudio.ai/v1)
 - 5.1 Chat Completions
 - 5.2 Model Listing
6. Engine Deployment Management
 - 6.1 Create Endpoint
 - 6.2 Scale / Terminate Endpoint
 - 6.3 Endpoint Health
7. Account & Billing APIs
8. Database Schema (Supabase)
9. UI Component → API Mapping
10. Error Codes Reference
11. Mock Data Locations & Migration Guide

1. Overview & Architecture

SLM Studio is a full-stack platform for fine-tuning Small Language Models (SLMs). The UI is built with React 18 + Vite, backed by Supabase for user data, and connected to an external GPU-powered fine-tuning engine for training, evaluation, and deployment.

Layer	Technology	Responsibility
Browser / UI	React 18 + shadcn/ui	User interaction, form management, visualisation
Supabase BaaS	PostgreSQL + Auth + RLS	User accounts, project metadata, API keys, analytics events
Fine-Tune Engine	engine.slmstudio.ai/v1	GPU training, HPO, SSE progress stream, evaluation
Inference API	api.slmstudio.ai/v1	OpenAI-compatible chat completions for deployed models
Deployment Mgr	engine.slmstudio.ai/v1	Endpoint lifecycle: create, scale, health, terminate

Training Job State Flow

queued → training → evaluating → completed ↓ ↓ failed failed

⚠ ⚠ All training operations are asynchronous. Poll GET /jobs/{id} or subscribe to GET /jobs/{id}/stream (SSE) to track progress.

2. Authentication

2.1 Supabase Auth (Internal)

Supabase manages user registration and login via JWT tokens stored in the browser. All Supabase table operations use Row-Level Security (RLS) so users can only access their own rows.

Method	Endpoint	Description
POST	POST /auth/v1/signup	Register new account (email + password)
POST	POST /auth/v1/token	Login — returns access_token (JWT)
POST	POST /auth/v1/logout	Invalidate current session
GET	GET /auth/v1/user	Return current user profile

2.2 SLM API Keys (External Engine & Inference)

External calls to **engine.slmstudio.ai/v1** and **api.slmstudio.ai/v1** use Bearer token authentication with SLM-format API keys.

Authorization: Bearer sk-slm-{prefix}-{random4}

⚠️ API keys are stored hashed in Supabase. The plain-text key is shown only once at creation. Key format: sk-slm-{first4chars}-{4-char-random}. Example: sk-slm-mypr-a3f9

Key Scopes

Scope	Allowed Operations
inference	POST /chat/completions, GET /models
training	POST /datasets, POST /jobs, GET /jobs/*
deployment	POST /endpoints, PATCH /endpoints/{id}, DELETE /endpoints/{id}
read-only	GET operations only (no mutations)
admin	All operations including billing and HPO

3. Internal Supabase APIs

3.1 Projects API

⚠ ⚠ These functions call the Supabase JavaScript client directly. No HTTP base URL is needed; authentication is via the active Supabase session.

listProjects()

[GET] /rest/v1/projects — Return all projects owned by current user

Field	Type	Description
id	uuid	Primary key
name	string	Project display name
description	string	Task prompt / description
task_type	string	classification generation summarisation translation qa code
base_model	string	Base model identifier (e.g. mistral-7b)
status	string	draft training completed failed
epochs	integer	Number of training epochs
learning_rate	float	Learning rate (e.g. 0.0001)
dataset_size	integer	Estimated dataset row count
created_at	timestamp	ISO 8601 creation timestamp
updated_at	timestamp	ISO 8601 last-updated timestamp

createProject(input)

[POST] /rest/v1/projects — Insert a new project row

Parameter	Type	Required	Description
name	string	Yes	Project name (max 120 chars)
description	string	Yes	Task prompt
taskType	TaskType	Yes	One of: classification generation summarisation translation qa code
baseModel	BaseModel	Yes	One of supported base model identifiers
epochs	integer	No	Default auto-computed by autoTuneParams()
learningRate	float	No	Default auto-computed
datasetSize	integer	No	Estimated number of dataset rows

updateProject(id, patch)

[PATCH] /rest/v1/projects?id=eq.{id} — Partial update of a project

Parameter	Type	Required	Description
id	uuid	Yes	Project UUID
patch	object	Yes	Partial ProjectRow fields to update

deleteProject(id)

[DELETE] /rest/v1/projects?id=eq.{id} — Delete a project

Parameter	Type	Required	Description
id	uuid	Yes	Project UUID

3.2 Models API

⚠ ⚠ Returns trained_models rows joined with associated project metadata.

listModels()

[GET] /rest/v1/trained_models — Return all trained models for current user

Field	Type	Description
id	uuid	Primary key
project_id	uuid	Parent project UUID
name	string	Model display name
task_type	string	Task category
base_model	string	Foundation model used
status	string	training completed failed deployed
accuracy	float	Evaluation accuracy (0-1)
f1_score	float	F1 score (0-1)
inference_speed	float	Tokens per second
model_size_mb	integer	Model file size in MB
created_at	timestamp	ISO 8601

getModel(id)

[GET] /rest/v1/trained_models?id=eq.{id} — Return a single model by ID

Parameter	Type	Required	Description
id	uuid	Yes	Model UUID

3.3 Datasets API

⚠ ⚠ Dataset rows are metadata only. Actual file content is stored on the Engine.

listDatasets()

[GET] /rest/v1/datasets — Return all datasets for current user

Field	Type	Description
id	uuid	Primary key
project_id	uuid	Associated project (nullable)
name	string	Dataset name
description	string	Optional description
rows	integer	Number of data rows
columns	integer	Number of columns/features
file_size	string	Human-readable size (e.g. 2.4 MB)
format	string	csv jsonl parquet txt
quality_score	float	0-100 auto quality score from Engine
created_at	timestamp	ISO 8601

3.4 Deployments API

⚠️ ⚠️ Endpoint status transitions: inactive → active → inactive. Hardware allocation is managed by the Engine.

listEndpoints()

[GET] /rest/v1/deployed_endpoints — Return all deployed endpoints for current user

Field	Type	Description
id	uuid	Primary key
model_id	uuid	Associated trained_models UUID
name	string	Endpoint display name
status	string	active inactive provisioning error
url	string	HTTPS endpoint URL for inference
region	string	Deployment region (e.g. us-east-1)
replicas	integer	Number of running replicas
requests_per_min	integer	Current RPS (from Engine metrics)
avg_latency_ms	integer	P50 latency from Engine
created_at	timestamp	ISO 8601

setEndpointStatus(id, status)

[PATCH] /rest/v1/deployed_endpoints?id=eq.{id} — Update endpoint status

Parameter	Type	Required	Description
id	uuid	Yes	Endpoint UUID
status	string	Yes	active inactive

3.5 API Keys API

⚠️ ⚠️ Plain-text key is returned only at creation. Store it immediately; it cannot be retrieved after the initial response.

listApiKeys()

[GET] /rest/v1/api_keys — Return all API keys for current user (hashed values only)

Field	Type	Description
id	uuid	Primary key
name	string	User-given label
key_prefix	string	First 12 chars for display (e.g. sk-slm-mypr-)
key_hash	string	SHA-256 hash of full key
scopes	string[]	Array of granted scopes
last_used	timestamp	Last successful API call timestamp (nullable)
created_at	timestamp	ISO 8601
expires_at	timestamp	Expiry timestamp (nullable = no expiry)

createApiKey(name)

[POST] /rest/v1/api_keys — Generate a new API key

Parameter	Type	Required	Description
name	string	Yes	Display label for the key (max 60 chars)

revokeApiKey(id)

[DELETE] /rest/v1/api_keys?id=eq.{id} — Hard-delete an API key

Parameter	Type	Required	Description
id	uuid	Yes	API key UUID

3.6 Analytics API

⚠️ ⚠️ Analytics events are written by the Inference API on each request. The UI reads aggregate views only; individual event rows may be large.

listCallEvents(range)

[GET] /rest/v1/api_call_events — Return paginated call events within a time range

Parameter	Type	Required	Description
range	string	No	7d 30d 90d (default: 30d)
endpoint_id	uuid	No	Filter by endpoint UUID
model_id	uuid	No	Filter by model UUID

summarize() / bucketByTime() / endpointStats()

Helper functions computed client-side from listCallEvents() data. bucketByTime() aggregates events by hour/day. endpointStats() computes p50/p95/p99 latency percentiles and error rates per endpoint.

Field	Type	Description
total_calls	integer	Total API calls in range
total_tokens	integer	Total tokens processed
avg_latency_ms	float	Mean latency across all calls
p50_latency_ms	float	Median (50th percentile) latency
p95_latency_ms	float	95th percentile latency
p99_latency_ms	float	99th percentile latency
error_rate	float	Fraction of requests with status >= 400
success_rate	float	Fraction of requests with status 200

4. External Fine-Tuning Engine APIs

Base URL: engine.slmstudio.ai/v1

The fine-tuning engine is a GPU-backed service that handles all computationally intensive operations: dataset ingestion, model training, hyperparameter optimisation, evaluation, and deployment management. All requests require a valid SLM API key with appropriate scope.

⚠️ The engine uses asynchronous job queuing. Long-running operations return a `job_id` immediately; results are retrieved via polling or SSE streaming.

4.1 Dataset Upload & Analysis

Upload Dataset

[POST] `/datasets` — Upload a new dataset file for analysis

Parameter	Type	Required	Description
<code>file</code>	File	Yes	Multipart file upload (CSV, JSONL, Parquet, TXT)
<code>project_id</code>	string	Yes	Supabase project UUID to associate with
<code>name</code>	string	Yes	Human-readable dataset name
<code>description</code>	string	No	Optional description
<code>task_type</code>	string	No	Hint for quality analysis

Field	Type	Description
<code>dataset_id</code>	string	Engine-assigned dataset ID (store in Supabase datasets table)
<code>rows</code>	integer	Detected row count
<code>columns</code>	integer	Detected column count
<code>file_size</code>	string	Human-readable size
<code>format</code>	string	Detected format
<code>quality_score</code>	float	0-100 quality score (completeness, balance, etc.)
<code>analysis</code>	object	Detailed analysis: <code>class_distribution</code> , <code>missing_values</code> , <code>token_lengths</code>
<code>warnings</code>	string[]	Quality warnings (e.g. 'class imbalance detected')

Get Dataset Analysis

[GET] `/datasets/{dataset_id}/analysis` — Retrieve full quality analysis report

Parameter	Type	Required	Description
<code>dataset_id</code>	string	Yes	Engine dataset ID
<code>include_sample</code>	boolean	No	Include 10-row preview (default false)

4.2 Training Job Lifecycle

Create Training Job

[POST] /jobs — Queue a new fine-tuning job

Parameter	Type	Required	Description
project_id	string	Yes	Supabase project UUID
dataset_id	string	Yes	Engine dataset ID from upload step
base_model	string	Yes	Foundation model (e.g. mistral-7b, llama-3-8b)
task_type	string	Yes	classification generation summarisation translation qa code
epochs	integer	No	Override auto-tuned epochs (1-50)
learning_rate	float	No	Override auto-tuned LR (1e-6 to 1e-2)
batch_size	integer	No	Override auto-tuned batch size (4-128)
warmup_steps	integer	No	LR warmup steps (default: 100)
weight_decay	float	No	L2 regularisation (default: 0.01)
lora_rank	integer	No	LoRA rank for PEFT training (default: 8)
use_hpo	boolean	No	Run HPO before training (default: false)
hpo_trials	integer	No	Number of HPO trials if use_hpo=true (default: 10)

Field	Type	Description
job_id	string	Engine job UUID
status	string	queued
estimated_time	integer	Estimated training seconds
queue_position	integer	Position in GPU queue
created_at	timestamp	ISO 8601

Get Job Status

[GET] /jobs/{job_id} — Return current status and metrics for a job

Field	Type	Description
job_id	string	Engine job UUID
status	string	queued training evaluating completed failed
current_epoch	integer	Current epoch number
total_epochs	integer	Total epochs planned
train_loss	float	Latest training loss
eval_loss	float	Latest evaluation loss
accuracy	float	Latest evaluation accuracy
elapsed_seconds	integer	Elapsed training time
model_id	string	Engine model ID (set when completed)
error	string	Error message if status=failed

Cancel Job

[DELETE] /jobs/{job_id} — Cancel a queued or running job

Parameter	Type	Required	Description
job_id	string	Yes	Engine job UUID

4.3 Training Progress Stream (SSE)

[GET] /jobs/{job_id}/stream — Server-Sent Events stream of training progress

Subscribe with EventSource. Each event has type **progress** or **log**:

```
const es = new EventSource(`https://engine.slmstudio.ai/v1/jobs/${jobId}/stream`, {
  headers: { Authorization: `Bearer ${apiKey}` } });
es.addEventListener('progress', (e) => {
  const d = JSON.parse(e.data); // d.epoch, d.loss, d.accuracy, d.elapsed_seconds
  updateProgressBar(d.epoch / d.total_epochs);
});
es.addEventListener('log', (e) => {
  appendLog(JSON.parse(e.data).message);
});
es.addEventListener('complete', (e) => {
  const d = JSON.parse(e.data); // d.model_id, d.final_accuracy, d.final_loss
  es.close();
});
```

⚠ ⚠ The current codebase (useTrainingSimulator.ts) uses a fake setTimeout loop. Replace with the EventSource pattern above to receive real training progress.

SSE Event Payload Fields

Field	Type	Description
type	string	progress log complete error
epoch	integer	Current epoch (progress events)
total_epochs	integer	Total epochs (progress events)
loss	float	Training loss at this step
eval_loss	float	Validation loss (if eval step)
accuracy	float	Validation accuracy (if eval step)
elapsed_seconds	integer	Elapsed seconds since job start
message	string	Human-readable log line (log events)
model_id	string	Final model ID (complete event only)

4.4 Model Evaluation

Trigger Evaluation

[POST] /models/{model_id}/evaluate — Run full evaluation on a held-out test set

Parameter	Type	Required	Description
model_id	string	Yes	Engine model ID
dataset_id	string	No	Evaluation dataset (defaults to 20% split from training set)
metrics	array	No	List of metrics: accuracy, f1, bleu, rouge, perplexity

Field	Type	Description
eval_id	string	Evaluation run UUID
accuracy	float	Accuracy score (0-1)
f1_score	float	Macro F1 score
precision	float	Macro precision
recall	float	Macro recall
bleu	float	BLEU score (generation tasks)
rouge_l	float	ROUGE-L score (summarisation tasks)
perplexity	float	Perplexity (language modelling tasks)
confusion_matrix	object	Class-level confusion matrix (classification tasks)
inference_speed	float	Tokens per second on eval hardware
evaluated_at	timestamp	ISO 8601

⚠ ⚠ ProjectDetail.tsx currently computes metrics from a hash of the project ID (lines 77-86). Replace with this endpoint to get real evaluation scores.

4.5 Hyperparameter Optimisation (HPO)

HPO runs multiple short training trials to find the best hyperparameter combination. Results are stored and can be applied automatically to the full training run.

Create HPO Job

[POST] /hpo — Queue a hyperparameter search job

Parameter	Type	Required	Description
dataset_id	string	Yes	Engine dataset ID
base_model	string	Yes	Foundation model
task_type	string	Yes	Task category
n_trials	integer	No	Number of search trials (default: 10, max: 50)
search_space	object	No	Override default search space (lr, batch_size, epochs, lora_rank)
metric	string	No	Optimisation target: accuracy f1 loss (default: accuracy)

Field	Type	Description
hpo_id	string	HPO job UUID
status	string	queued running completed
best_trial	object	Best trial: {lr, batch_size, epochs, accuracy}
all_trials	array	All trial results sorted by metric descending
recommended_config	object	Ready-to-use config for POST /jobs

4.6 Auto-Tuning Parameters (Client-side Heuristic)

The client has a lightweight auto-tuning heuristic in **NewProject.tsx** that estimates hyperparameters from dataset size before the engine HPO runs:

Dataset Size (rows)	Epochs	Learning Rate	Batch Size
< 1,000	10	1e-4	8
1,000 – 5,000	5	2e-4	16
> 5,000	3	3e-4	32

⚠️ This heuristic uses files.length × 500 as a dataset size proxy. After engine integration, use the actual row count from POST /datasets response.

5. Inference API (api.slmstudio.ai/v1)

The Inference API is OpenAI-compatible, allowing any OpenAI SDK to be pointed at api.slmstudio.ai/v1 with a SLM API key.

5.1 Chat Completions

[POST] /chat/completions — Generate a chat response from a deployed SLM model

Parameter	Type	Required	Description
model	string	Yes	Deployed model name (from /models list)
messages	array	Yes	[[role: user assistant system, content: string]]
temperature	float	No	Sampling temperature 0-2 (default: 0.7)
max_tokens	integer	No	Maximum tokens to generate (default: 512)
stream	boolean	No	Enable streaming (default: false)
top_p	float	No	Nucleus sampling (default: 1.0)
stop	array	No	Stop sequences

Field	Type	Description
id	string	Response UUID
object	string	chat.completion
model	string	Model name used
choices	array	[[index, message: {role, content}, finish_reason]]
usage	object	{prompt_tokens, completion_tokens, total_tokens}
created	integer	Unix timestamp

⚠️ ⚠️ ChatPanel.tsx currently uses mockResponses array with setTimeout. Replace handleSend() with a fetch to this endpoint.

5.2 Model Listing

[GET] /models — List all models available for inference

Field	Type	Description
id	string	Model identifier
name	string	Display name
task_type	string	Task category
context_length	integer	Maximum context window in tokens
status	string	available loading offline
created	integer	Unix timestamp

6. Engine Deployment Management

Deployment operations create and manage GPU-backed inference endpoints. Endpoint metadata is mirrored to the Supabase deployed_endpoints table after creation.

6.1 Create Endpoint

[POST] /endpoints — Provision a new inference endpoint

Parameter	Type	Required	Description
model_id	string	Yes	Engine model ID (from completed training job)
name	string	Yes	Endpoint display name
region	string	No	us-east-1 us-west-2 eu-west-1 ap-southeast-1
replicas	integer	No	Initial replica count (default: 1, max: 8)
instance_type	string	No	gpu-small gpu-medium gpu-large (default: gpu-small)
autoscale	boolean	No	Enable auto-scaling (default: false)
min_replicas	integer	No	Minimum replicas for autoscaling (default: 1)
max_replicas	integer	No	Maximum replicas for autoscaling (default: 4)

Field	Type	Description
endpoint_id	string	Engine endpoint UUID (store as deployed_endpoints.id)
url	string	HTTPS inference URL
status	string	provisioning
region	string	Provisioned region
estimated_ready_seconds	integer	ETA for endpoint to become active

6.2 Scale / Terminate Endpoint

Scale Endpoint

[PATCH] /endpoints/{endpoint_id} — Change replica count or instance type

Parameter	Type	Required	Description
endpoint_id	string	Yes	Engine endpoint UUID
replicas	integer	No	New replica count
instance_type	string	No	New instance type

Terminate Endpoint

[DELETE] /endpoints/{endpoint_id} — Shut down and deallocate an endpoint

Parameter	Type	Required	Description
endpoint_id	string	Yes	Engine endpoint UUID

6.3 Endpoint Health

[GET] /endpoints/{endpoint_id}/health — Return real-time metrics for an endpoint

Field	Type	Description
status	string	active loading error offline

Field	Type	Description
replicas_ready	integer	Number of healthy replicas
requests_per_min	integer	Requests in last 60 seconds
avg_latency_ms	integer	Rolling P50 latency
p95_latency_ms	integer	Rolling P95 latency
error_rate	float	Fraction of 5xx responses in last 5 min
gpu_utilisation	float	Average GPU utilisation (0-1)
checked_at	timestamp	ISO 8601

⚠ ⚠ Deployment.tsx uses Math.random() for timeline values on every render. Replace mockUsageTimeline with periodic calls to this health endpoint.

7. Account & Billing APIs

⚠ Billing is managed by the Engine service. Supabase stores a cached copy of `credits_remaining` and `plan_tier` for fast UI display.

7.1 Get Account Info

[GET] `/account` — Return plan, credits, and usage summary

Field	Type	Description
<code>user_id</code>	string	Supabase user UUID
<code>plan_tier</code>	string	free starter pro enterprise
<code>credits_remaining</code>	float	GPU-compute credits remaining
<code>credits_used</code>	float	Credits used this billing period
<code>training_jobs_used</code>	integer	Jobs consumed this period
<code>training_jobs_limit</code>	integer	Plan job limit (null = unlimited)
<code>endpoints_active</code>	integer	Currently active endpoint count
<code>endpoints_limit</code>	integer	Plan endpoint limit
<code>renews_at</code>	timestamp	Next billing period start

7.2 Get Usage History

[GET] `/account/usage` — Return credit consumption history by job/endpoint

Parameter	Type	Required	Description
<code>range</code>	string	No	7d 30d 90d (default: 30d)
<code>type</code>	string	No	training inference all (default: all)

Field	Type	Description
<code>events</code>	array	Array of {timestamp, type, amount, description, job_id}
<code>total_consumed</code>	float	Total credits consumed in range
<code>by_type</code>	object	{training: float, inference: float}

8. Database Schema (Supabase)

All tables use Supabase Row-Level Security. Rows are automatically filtered to the authenticated user via `auth.uid()`.

TABLE: projects

Column	Type	Constraints / Notes
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()
user_id	uuid	NOT NULL FK <code>auth.users(id)</code> ON DELETE CASCADE
name	text	NOT NULL CHECK (length(name) <= 120)
description	text	
task_type	text	NOT NULL CHECK (task_type IN ('classification','generation','summarisation','translation',
base_model	text	NOT NULL
status	text	DEFAULT 'draft' CHECK (status IN ('draft','training','completed','failed'))
epochs	integer	DEFAULT 5
learning_rate	numeric(10,8)	DEFAULT 0.0001
dataset_size	integer	DEFAULT 0
created_at	timestamptz	DEFAULT now()
updated_at	timestamptz	DEFAULT now()

TABLE: trained_models

Column	Type	Constraints / Notes
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()
user_id	uuid	NOT NULL FK <code>auth.users(id)</code> ON DELETE CASCADE
project_id	uuid	FK <code>projects(id)</code> ON DELETE SET NULL
name	text	NOT NULL
task_type	text	NOT NULL
base_model	text	NOT NULL
status	text	DEFAULT 'training' CHECK (status IN ('training','completed','failed','deployed'))
accuracy	numeric(5,4)	NULLABLE — from Engine evaluation
f1_score	numeric(5,4)	NULLABLE
inference_speed	numeric(8,2)	NULLABLE — tokens/sec from Engine
model_size_mb	integer	NULLABLE
engine_model_id	text	Engine-side model UUID (set after training completes)
created_at	timestamptz	DEFAULT now()

TABLE: datasets

Column	Type	Constraints / Notes
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()
user_id	uuid	NOT NULL FK <code>auth.users(id)</code> ON DELETE CASCADE
project_id	uuid	NULLABLE FK <code>projects(id)</code> ON DELETE SET NULL

Column	Type	Constraints / Notes
name	text	NOT NULL
description	text	
rows	integer	DEFAULT 0
columns	integer	DEFAULT 0
file_size	text	Human-readable (e.g. '2.4 MB')
format	text	csv jsonl parquet txt
quality_score	numeric(5,2)	0-100 from Engine analysis
engine_dataset_id	text	Engine-side dataset UUID
created_at	timestampz	DEFAULT now()

TABLE: deployed_endpoints

Column	Type	Constraints / Notes
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()
user_id	uuid	NOT NULL FK <code>auth.users(id)</code> ON DELETE CASCADE
model_id	uuid	FK <code>trained_models(id)</code> ON DELETE CASCADE
name	text	NOT NULL
status	text	DEFAULT 'inactive' CHECK (status IN ('active','inactive','provisioning','error'))
url	text	HTTPS endpoint URL
region	text	Deployment region
replicas	integer	DEFAULT 1
engine_endpoint_id	text	Engine-side endpoint UUID
requests_per_min	integer	Synced from Engine health check
avg_latency_ms	integer	Synced from Engine health check
created_at	timestampz	DEFAULT now()

TABLE: api_keys

Column	Type	Constraints / Notes
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()
user_id	uuid	NOT NULL FK <code>auth.users(id)</code> ON DELETE CASCADE
name	text	NOT NULL User-given label
key_prefix	text	First 12 chars for display
key_hash	text	SHA-256 hash of full key NOT NULL
scopes	text[]	Array of scopes
last_used	timestampz	NULLABLE
expires_at	timestampz	NULLABLE — null means no expiry
created_at	timestampz	DEFAULT now()

TABLE: api_call_events

Column	Type	Constraints / Notes
id	uuid	PRIMARY KEY DEFAULT gen_random_uuid()

Column	Type	Constraints / Notes
user_id	uuid	NOT NULL FK auth.users(id) ON DELETE CASCADE
endpoint_id	uuid	NULLABLE FK deployed_endpoints(id) ON DELETE SET NULL
model_id	uuid	NULLABLE FK trained_models(id) ON DELETE SET NULL
status_code	integer	HTTP status of the inference request
latency_ms	integer	Total request latency
prompt_tokens	integer	Input token count
completion_tokens	integer	Output token count
created_at	timestamptz	DEFAULT now()

9. UI Component → API Mapping

This table shows which UI components need real API calls and the current status of each. Components marked with a warning currently use mock/simulated data.

UI Component	Data Source	Required API	Status
Dashboard / StatsCards	Supabase projects, models	listProjects(), listModels()	✓ Real
Dashboard / StatsCards (credits)	Engine billing	GET /account	✗ Missing
RecentProjects	Supabase projects	listProjects()	✓ Real
ActivityChart	Supabase api_call_events	bucketByTime()	✓ Real
NewProject / autoTune	Client heuristic	POST /hpo (if use_hpo=true)	⚠ Mock
TrainingMonitor / progress	useTrainingSimulator.ts	GET /jobs/{id}/stream (SSE)	⚠ Mock
TrainingMonitor / logs	mockTrainingLog	SSE log events	⚠ Mock
TrainingMonitor / loss curve	mockLossCurve	SSE progress events	⚠ Mock
ProjectDetail / metrics	Hash of project.id	GET /models/{id}/evaluate	⚠ Mock
ProjectDetail / versions	mockVersionHistory	GET /jobs?project_id={id}	⚠ Mock
Playground / ChatPanel	mockResponses + setTimeout	POST /chat/completions	⚠ Mock
Deployment / usage chart	Math.random() each render	GET /endpoints/{id}/health	⚠ Mock
Models / trained_models	Supabase trained_models	listModels()	✓ Real
Analytics / latency	Supabase api_call_events	endpointStats()	✓ Real
Settings / API Keys	Supabase api_keys	listApiKeys(), createApiKey()	✓ R
Leaderboard	Supabase trained_models	listModels() (sorted)	✓ Real
DatasetInsights	Supabase datasets	GET /datasets/{id}/analysis	✗ Missing

10. Error Codes Reference

All APIs (Supabase and Engine) return standard HTTP status codes. Engine errors include a machine-readable code field.

HTTP	Engine Code	Meaning	Action
400	INVALID_PARAMS	Missing or invalid request parameters	Check required fields
401	UNAUTHORIZED	Missing or invalid Bearer token	Check API key / session
403	FORBIDDEN	Valid token but insufficient scope	Request wider-scope key
404	NOT_FOUND	Resource does not exist	Verify IDs
409	CONFLICT	Duplicate resource or state conflict	Check existing records
413	PAYLOAD_TOO_LARGE	File exceeds 500 MB limit	Split or compress dataset
422	VALIDATION_ERROR	Business rule violation	See error.details
429	RATE_LIMITED	Too many requests	Backoff and retry
402	INSUFFICIENT_CREDITS	Not enough GPU credits	Top up or upgrade plan
503	ENGINE_UNAVAILABLE	No GPU capacity available	Retry with exponential backoff
500	INTERNAL_ERROR	Unexpected engine error	Contact support with job_id

Retry Strategy

For 429 and 503 errors, use exponential backoff: wait 1s, 2s, 4s, 8s, 16s before giving up. Include a Idempotency-Key header on POST requests to safely retry.

11. Mock Data Locations & Migration Guide

The following source files contain hardcoded/simulated data that must be replaced with real API calls before production deployment.

[src/components/playground/ChatPanel.tsx](#)

Current: mockResponses array + setTimeout in handleSend()

Fix: Replace with fetch('https://api.slmstudio.ai/v1/chat/completions', {...})

[src/hooks/useTrainingSimulator.ts](#)

Current: setTimeout loop every 2000ms, fake progress saved to Supabase

Fix: Replace with EventSource subscription to GET /jobs/{id}/stream

[src/pages/ProjectDetail.tsx \(lines 77-86\)](#)

Current: Metrics computed as $85 + ((\text{seed} + \text{offset}) \% 12)$ from project.id hash

Fix: Replace with GET /models/{engine_model_id}/evaluate response

[src/pages/ProjectDetail.tsx](#)

Current: mockVersionHistory — hardcoded array of fake versions

Fix: Replace with GET /jobs?project_id={id} (list training job history)

[src/pages/TrainingMonitor.tsx](#)

Current: mockPipelineSteps, mockTrainingLog, mockLossCurve, mockComparisonResults

Fix: Replace with SSE stream events; epoch hardcoded as '5 / 8'

[src/data/deploymentMockData.ts \(lines 72-77\)](#)

Current: mockUsageTimeline — Math.random() called each render

Fix: Replace with periodic GET /endpoints/{id}/health calls (poll every 30s)

[src/components/dashboard/StatsCards.tsx](#)

Current: Missing credits_remaining and plan_tier — no billing API call

Fix: Add GET /account call; display credits_remaining and plan_tier

[src/pages/DatasetInsights.tsx](#)

Current: Dataset quality shown from Supabase metadata only

Fix: Fetch full analysis from GET /datasets/{engine_dataset_id}/analysis

⚠️ ⚠️ After migration, remove src/data/trainingMockData.ts, src/data/deploymentMockData.ts, and the mockResponses array in ChatPanel.tsx.